

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <chrono>

using namespace std;

void display_data(const vector<int>& arr) {
    for (size_t i = 0; i < arr.size(); i++) {
        cout << arr[i];
        if (i < arr.size() - 1) {
            cout << ", ";
        }
    }
    cout << endl;
}

int find_data(const vector<int>& arr, int target) {
    for (size_t i = 0; i < arr.size(); i++) {
        if (arr[i] == target) {
            return i;
        }
    }
    return -1;
}

void insert_data(vector<int>& arr, int data, size_t index) {
    arr.insert(arr.begin() + index, data);
}

void delete_data(vector<int>& arr, int target) {
    arr.erase(remove(arr.begin(), arr.end(), target), arr.end());
}

vector<int> quick_sort(const vector<int>& arr) {
    if (arr.size() <= 1) {
        return arr;
    }
    int pivot = arr[arr.size() / 2];
    vector<int> left, middle, right;
    for (int x : arr) {
        if (x < pivot)
            left.push_back(x);
        else if (x == pivot)
            middle.push_back(x);
        else
            right.push_back(x);
    }
    vector<int> sorted_arr = quick_sort(left);
    sorted_arr.insert(sorted_arr.end(), middle.begin(), middle.end());
    vector<int> right_sorted = quick_sort(right);
    sorted_arr.insert(sorted_arr.end(), right_sorted.begin(), right_sorted.end());
    return sorted_arr;
}

vector<int> exchange_sort(vector<int> arr) {
    for (size_t i = 0; i < arr.size(); i++) {
        for (size_t j = i + 1; j < arr.size(); j++) {
            if (arr[i] > arr[j]) {
                swap(arr[i], arr[j]);
            }
        }
    }
    return arr;
}

int main() {
    vector<int> data = {45, 16, 10, 25, 11, 80, 62, 18, 20, 37};

    // a. Display data
    cout << "Output Point A." << endl;
    display_data(data);

    // b. Find data 18
    cout << "Output Point B." << endl;
    int target = 18;
    int index = find_data(data, target);
    if (index == -1) {
        cout << "Data " << target << " found at index array ke " << index << " atau (Data[" << index << "]=" << target << ")" << endl;
    } else {
        cout << "Data " << target << " not found." << endl;
    }

    // c. Insert data 100 at index 5
    cout << "Output Point C." << endl;
    cout << "Sebelum di insert adalah ";
    display_data(data);
    insert_data(data, 100, 5);
    cout << "Setelah di insertkan data 100 adalah ";
    display_data(data);

    // d. Delete data 62
    cout << "Output Point D." << endl;
    cout << "Data Sebelum dihapus data 62 adalah ";
    display_data(data);
    delete_data(data, 62);
    cout << "Data Setelah dihapus data 62 adalah ";
    display_data(data);

    // e. Sort data with Quick-Sort and Exchange-Sort
    cout << "Output Point E." << endl;
    cout << "Data Sebelum diurutkan adalah ";
    display_data(data);

    vector<int> sorted_data_quick = quick_sort(data);
    cout << "Data Setelah diurutkan dengan metode Quick-Sort adalah ";
    display_data(sorted_data_quick);

    vector<int> sorted_data_exchange = exchange_sort(data);
    cout << "Data Setelah diurutkan dengan metode Exchange-Sort adalah ";
    display_data(sorted_data_exchange);

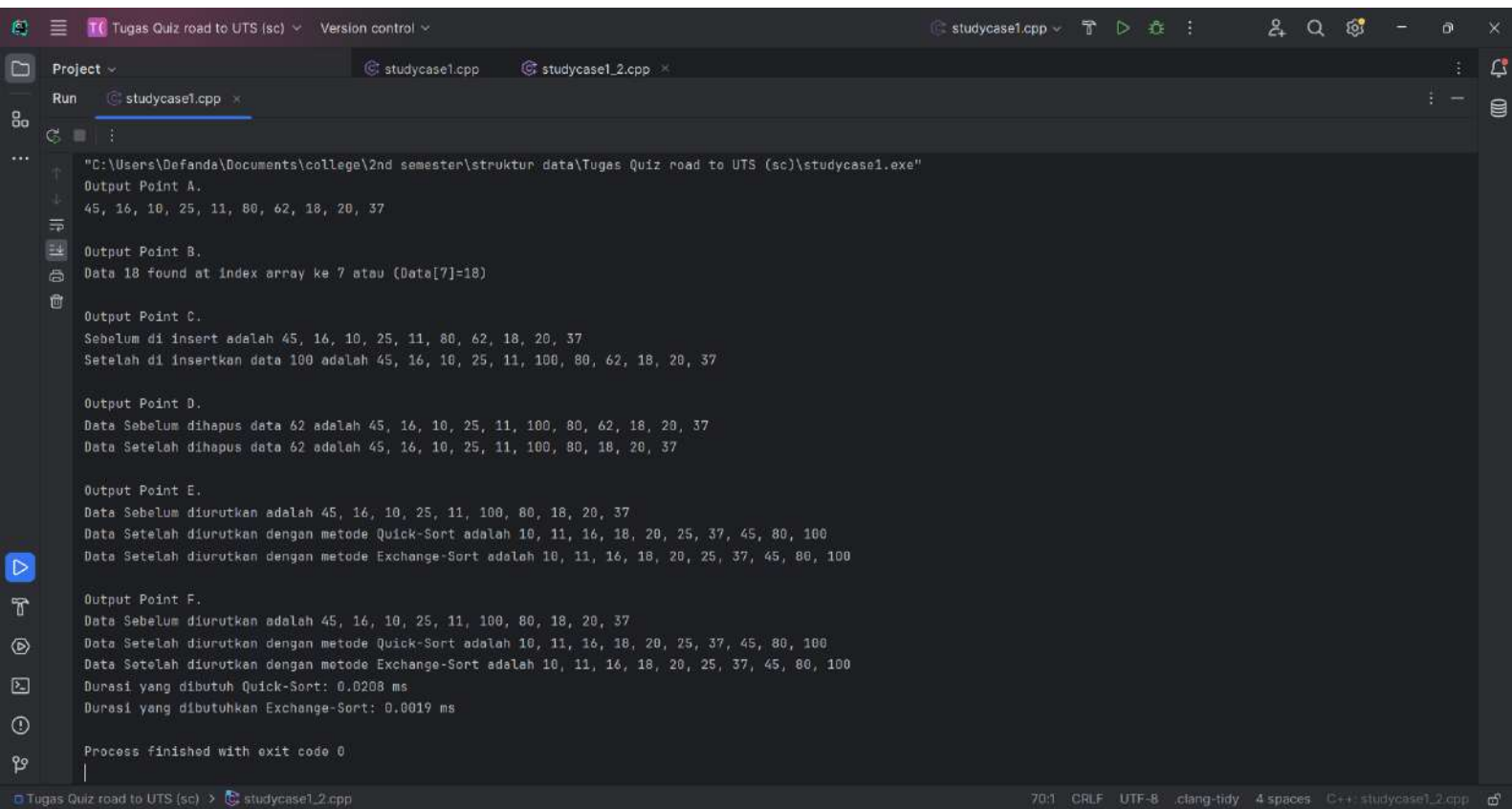
    // f. Compare sorting methods
    cout << "Output Point F." << endl;
    cout << "Data Sebelum diurutkan adalah ";
    display_data(data);
    cout << "Data Setelah diurutkan dengan metode Quick-Sort adalah ";
    display_data(sorted_data_quick);
    cout << "Data Setelah diurutkan dengan metode Exchange-Sort adalah ";
    display_data(sorted_data_exchange);

    // Measure execution time for Quick-Sort
    auto start_quick = chrono::steady_clock::now();
    sorted_data_quick = quick_sort(data); // Reuse existing variable
    auto end_quick = chrono::steady_clock::now();
    cout << "Durasi yang dibutuhkan Quick-Sort: " << chrono::duration<double, milli>(end_quick - start_quick).count() << " ms" << endl;

    // Measure execution time for Exchange-Sort
    auto start_exchange = chrono::steady_clock::now();
    sorted_data_exchange = exchange_sort(data); // Reuse existing variable
    auto end_exchange = chrono::steady_clock::now();
    cout << "Durasi yang dibutuhkan Exchange-Sort: " << chrono::duration<double, milli>(end_exchange - start_exchange).count() << " ms" << endl;

    return 0;
}
```

output nomor 1



```
"C:\Users\Defanda\Documents\college\2nd semester\struktur_data\Tugas Quiz road to UTS (sc)\studycase1.exe"
Output Point A.
45, 16, 10, 25, 11, 80, 62, 18, 20, 37

Output Point B.
Data 18 found at index array ke 7 atau (Data[7]=18)

Output Point C.
Sebelum di insert adalah 45, 16, 10, 25, 11, 80, 62, 18, 20, 37
Setelah di insertkan data 100 adalah 45, 16, 10, 25, 11, 100, 80, 62, 18, 20, 37

Output Point D.
Data Sebelum dihapus data 62 adalah 45, 16, 10, 25, 11, 100, 80, 62, 18, 20, 37
Data Setelah dihapus data 62 adalah 45, 16, 10, 25, 11, 100, 80, 18, 20, 37

Output Point E.
Data Sebelum diurutkan adalah 45, 16, 10, 25, 11, 100, 80, 18, 20, 37
Data Setelah diurutkan dengan metode Quick-Sort adalah 10, 11, 16, 18, 20, 25, 37, 45, 80, 100
Data Setelah diurutkan dengan metode Exchange-Sort adalah 10, 11, 16, 18, 20, 25, 37, 45, 80, 100

Output Point F.
Data Sebelum diurutkan adalah 45, 16, 10, 25, 11, 100, 80, 18, 20, 37
Data Setelah diurutkan dengan metode Quick-Sort adalah 10, 11, 16, 18, 20, 25, 37, 45, 80, 100
Data Setelah diurutkan dengan metode Exchange-Sort adalah 10, 11, 16, 18, 20, 25, 37, 45, 80, 100
Durasi yang butuh Quick-Sort: 0.0208 ms
Durasi yang dibutuhkan Exchange-Sort: 0.0019 ms

Process finished with exit code 0
```

```
#include <iostream>
#include <iomanip>
#include <vector>
#include <string>

using namespace std;

struct Mahasiswa {
    char nama[31]; // Nama up to 30 characters
    char nim[11];  // NIM up to 10 characters
    float nilai;   // Nilai Struktur Data as float
    char grade;    // Grade as char
};

int main() {
    vector<Mahasiswa> dataMahasiswa; // Vector to store Mahasiswa data
    char tambahLagi;

    do {
        cout << "======" << endl;
        Mahasiswa mahasiswa;

        cout << "Nama (max 30 characters): ";
        cin.ignore();
        cin.getline(mahasiswa.nama, 31); // Read up to 30 characters

        cout << "NIM (max 10 characters): ";
        cin.getline(mahasiswa.nim, 11); // Read up to 10 characters

        cout << "Nilai Struktur Data: ";
        cin >> mahasiswa.nilai;

        if (mahasiswa.nilai >= 80 && mahasiswa.nilai <= 100) {
            mahasiswa.grade = 'A';
        } else if (mahasiswa.nilai >= 70 && mahasiswa.nilai <= 79) {
            mahasiswa.grade = 'B';
        } else if (mahasiswa.nilai >= 60 && mahasiswa.nilai <= 69) {
            mahasiswa.grade = 'C';
        } else if (mahasiswa.nilai >= 50 && mahasiswa.nilai <= 59) {
            mahasiswa.grade = 'D';
        } else {
            mahasiswa.grade = 'E';
        }

        // Add the mahasiswa data to the vector
        dataMahasiswa.push_back(mahasiswa);

        cout << "Tambah data? [y/n]: ";
        cin >> tambahLagi;
    } while (tambahLagi == 'y' || tambahLagi == 'Y');

    // Display the table
    cout << left << setw(30) << "Nama" << "|"
        << setw(10) << "NIM" << "|"
        << setw(20) << "Nilai Struktur Data" << "|"
        << setw(5) << "Grade" << "|" << endl;
    cout << setfill('-') << setw(68) << "-" << setfill(' ') << endl;

    float totalNilai = 0;
    for (const Mahasiswa &mahasiswa : dataMahasiswa) {
        cout << left << setw(30) << mahasiswa.nama << "|"
            << setw(10) << mahasiswa.nim << "|"
            << setw(20) << fixed << setprecision(2) << mahasiswa.nilai << "|"
            << setw(5) << mahasiswa.grade << "|" << endl;
        totalNilai += mahasiswa.nilai;
    }

    // Calculate the average
    float rataRata = totalNilai / dataMahasiswa.size();

    // Display the average
    cout << setfill('-') << setw(68) << "-" << setfill(' ') << endl;
    cout << "Rata-rata Nilai: " << fixed << setprecision(2) << rataRata << endl;

    return 0;
}
```

output nomor 2

/tmp/zhWo4L0y5d.o

=====

Nama (max 30 characters): Defanda

NIM (max 10 characters): 2023080125

Nilai Struktur Data: 90

Tambah data? [y/n]: y

=====

Nama (max 30 characters): Fahmi

NIM (max 10 characters): 2023080121

Nilai Struktur Data: 95

Tambah data? [y/n]: n

DAFTAR NAMA ANGGOTA KELOMPOK 1

Nama	NIM	Nilai Struktur Data	Grade
Defanda	2023080125	90.00	A
Fahmi	2023080121	95.00	A

Rata-rata Nilai: 92.50